Assignment 3 – Implementing a Multi-Layer Neural Network From Scratch

Submitters:
Omer Guz (209483718), Metar Bachar (206892317), Niv Naus (316197227)

GitHub notebooks:

- Task 2: https://github.com/omerguz/machine-learning-book/blob/two-hidden-layers/ch11/ch11_two_hidden_layers_task2.ipynb
- Tasks 3–4: https://github.com/omerguz/machine-learning-book/blob/two-hidden-layers/ch11/ch11_two_hidden_layers_task3%2B4.ipynb

1. Goal of the assignment

The assignment is based on Chapter 11 from Raschka et al. (2022), which implements a multi-layer neural network from scratch (NumPy), including forward pass, backpropagation, minibatch training, and evaluation.

We did the following:

1. Reviewed the Chapter 11 implementation.
2. Extended the code to support **two hidden layers** (Task 2).
3. Applied the two-layer model to **MNIST** using the **lecture "Solution 1: A plain deep NN" architecture** and evaluated performance with **Train(70%) / Test(30%)** and **macro AUC** (Task 3).
4. Compared results to (a) a **single-hidden-layer** baseline (same evaluation protocol) and (b) a **PyTorch fully-connected ANN** baseline (Task 4).

2. Background (what matters from Chapter 11)

Chapter 11 builds an ANN classifier "from scratch":

- Inputs are vectors (e.g., flattened image pixels).
- Hidden layers apply an activation (in the chapter: sigmoid).
- Training uses minibatch SGD, with manual gradient calculations (backprop).
- Targets are converted to **one-hot** vectors for loss computation.
- The model predicts class labels using argmax over the output scores.

3. Task 2 – Extending the original notebook to two hidden layers

Starting from the original Chapter 11 notebook, we modified the MLP implementation to include a second hidden layer.

Main changes:

- Added a new parameter num_hidden2.
- Added the second hidden layer parameters: weight_h2 and bias_h2.
- Updated forward() to compute:
  hidden1 → hidden2 → output
- Updated backward() to backpropagate gradients through both hidden layers.
- Updated the training loop to apply gradient updates to all layers.

Note: Task 2 keeps the original Chapter 11 output setup (sigmoid output), since the task is specifically about adding a second hidden layer.

4. Task 3 – MNIST using the lecture "Solution 1" architecture + evaluation protocol

For Task 3 we aligned exactly with the lecture slide "Solution 1: A plain deep NN":

Architecture (fully-connected):

- Input: 28×28 image → flatten to 784 features
- Hidden layer 1: 500 units + **sigmoid**
- Hidden layer 2: 500 units + **sigmoid**
- Output: 10 units + **softmax**

Training configuration (as in the lecture slides):

- Loss: **MSE**
- Optimizer: **SGD**, learning rate **0.1**
- Batch size: **100**
- Epochs: **20**

Evaluation protocol:

- Split: **Train 70% / Test 30%** (stratified)
- Metric required by assignment: **macro AUC (one-vs-rest, macro-average)**
  We compute AUC per class (class k vs all others), and then average across the 10 classes.

5. Task 4 – Comparisons

We compared:

1. **Single hidden layer (from scratch)** using the same evaluation protocol
2. **Two hidden layers (from scratch)** from Task 3
3. **Fully connected ANN in PyTorch** (same architecture and protocol)

Results (MNIST, Train 70% / Test 30%)

Model	Hidden Layers	Test Accuracy	Test Macro AUC (OVR, macro)	Test MSE
From-scratch (single hidden)	1×500	92.57%	0.9929	0.0115
From-scratch (lecture architecture)	500, 500	92.47%	0.9932	0.0114
PyTorch fully-connected baseline	500, 500	92.32%	0.9932	0.1171*

* The PyTorch MSE value is not directly comparable to the scratch MSE due to a different reduction/scaling choice for the MSE calculation (the accuracy and AUC are the meaningful comparison).

6. Discussion

- The **single-hidden** and **two-hidden** scratch models achieved very similar performance. Under the fixed setup (sigmoid + MSE + SGD + 20 epochs), adding depth did not give a clear accuracy gain.
- The **macro AUC** is high (~0.993) for both models, meaning the ranking/probability separation between classes is strong even when accuracy differences are small.
- The **PyTorch baseline** matched the scratch model's macro AUC and achieved a very close accuracy, which is a good consistency check that the scratch implementation behaves as expected.
- The lecture notes mention that the same general idea can reach higher accuracy (e.g., ~98%+) "with a lot of tuning". We did not tune beyond the requested configuration, and we kept the loss as **MSE** (as shown in the lecture slides), so our accuracy is naturally lower than heavily optimized modern MNIST setups (often cross-entropy + better optimizers).